

Chapter 2

Linear Algebra

Linear algebra is a branch of mathematics that is widely used throughout science and engineering. However, because linear algebra is a form of continuous rather than discrete mathematics, many computer scientists have little experience with it. A good understanding of linear algebra is essential for understanding and working with many machine learning algorithms, especially deep learning algorithms. We therefore precede our introduction to deep learning with a focused presentation of the key linear algebra prerequisites.

If you are already familiar with linear algebra, feel free to skip this chapter. If you have previous experience with these concepts but need a detailed reference sheet to review key formulas, we recommend *The Matrix Cookbook* ([Petersen and Pedersen, 2006](#)). If you have no exposure at all to linear algebra, this chapter will teach you enough to read this book, but we highly recommend that you also consult another resource focused exclusively on teaching linear algebra, such as [Shilov \(1977\)](#). This chapter will completely omit many important linear algebra topics that are not essential for understanding deep learning.

2.1 Scalars, Vectors, Matrices and Tensors

The study of linear algebra involves several types of mathematical objects:

- **Scalars:** A scalar is just a single number, in contrast to most of the other objects studied in linear algebra, which are usually arrays of multiple numbers. We write scalars in italics. We usually give scalars lower-case variable names. When we introduce them, we specify what kind of number they are. For

example, we might say “Let $s \in \mathbb{R}$ be the slope of the line,” while defining a real-valued scalar, or “Let $n \in \mathbb{N}$ be the number of units,” while defining a natural number scalar.

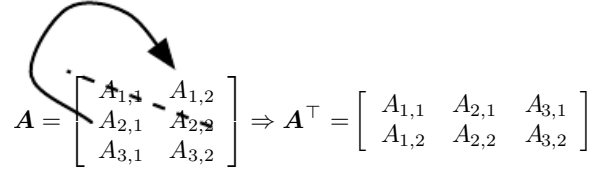
- **Vectors:** A vector is an array of numbers. The numbers are arranged in order. We can identify each individual number by its index in that ordering. Typically we give vectors lower case names written in bold typeface, such as $\mathbf{x}$. The elements of the vector are identified by writing its name in italic typeface, with a subscript. The first element of $\mathbf{x}$ is x_1 , the second element is x_2 and so on. We also need to say what kind of numbers are stored in the vector. If each element is in $\mathbb{R}$, and the vector has n elements, then the vector lies in the set formed by taking the Cartesian product of $\mathbb{R}$ n times, denoted as $\mathbb{R}^n$. When we need to explicitly identify the elements of a vector, we write them as a column enclosed in square brackets:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}. \quad (2.1)$$

We can think of vectors as identifying points in space, with each element giving the coordinate along a different axis.

Sometimes we need to index a set of elements of a vector. In this case, we define a set containing the indices and write the set as a subscript. For example, to access x_1 , x_3 and x_6 , we define the set $S = \{1, 3, 6\}$ and write $\mathbf{x}_S$. We use the $-$ sign to index the complement of a set. For example $\mathbf{x}_{-1}$ is the vector containing all elements of $\mathbf{x}$ except for x_1 , and $\mathbf{x}_{-S}$ is the vector containing all of the elements of $\mathbf{x}$ except for x_1 , x_3 and x_6 .

- **Matrices:** A matrix is a 2-D array of numbers, so each element is identified by two indices instead of just one. We usually give matrices upper-case variable names with bold typeface, such as $\mathbf{A}$. If a real-valued matrix $\mathbf{A}$ has a height of m and a width of n , then we say that $\mathbf{A} \in \mathbb{R}^{m \times n}$. We usually identify the elements of a matrix using its name in italic but not bold font, and the indices are listed with separating commas. For example, $A_{1,1}$ is the upper left entry of $\mathbf{A}$ and $A_{m,n}$ is the bottom right entry. We can identify all of the numbers with vertical coordinate i by writing a “:” for the horizontal coordinate. For example, $\mathbf{A}_{i,:}$ denotes the horizontal cross section of $\mathbf{A}$ with vertical coordinate i . This is known as the i -th **row** of $\mathbf{A}$. Likewise, $\mathbf{A}_{:,i}$ is



$$\mathbf{A} = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{bmatrix} \Rightarrow \mathbf{A}^\top = \begin{bmatrix} A_{1,1} & A_{2,1} & A_{3,1} \\ A_{1,2} & A_{2,2} & A_{3,2} \end{bmatrix}$$

Figure 2.1: The transpose of the matrix can be thought of as a mirror image across the main diagonal.

the i -th **column** of $\mathbf{A}$. When we need to explicitly identify the elements of a matrix, we write them as an array enclosed in square brackets:

$$\begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}. \quad (2.2)$$

Sometimes we may need to index matrix-valued expressions that are not just a single letter. In this case, we use subscripts after the expression, but do not convert anything to lower case. For example, $f(\mathbf{A})_{i,j}$ gives element (i, j) of the matrix computed by applying the function f to $\mathbf{A}$.

- **Tensors:** In some cases we will need an array with more than two axes. In the general case, an array of numbers arranged on a regular grid with a variable number of axes is known as a tensor. We denote a tensor named “A” with this typeface: $\mathbf{A}$. We identify the element of $\mathbf{A}$ at coordinates (i, j, k) by writing $A_{i,j,k}$.

One important operation on matrices is the **transpose**. The transpose of a matrix is the mirror image of the matrix across a diagonal line, called the **main diagonal**, running down and to the right, starting from its upper left corner. See figure 2.1 for a graphical depiction of this operation. We denote the transpose of a matrix $\mathbf{A}$ as $\mathbf{A}^\top$, and it is defined such that

$$(\mathbf{A}^\top)_{i,j} = A_{j,i}. \quad (2.3)$$

Vectors can be thought of as matrices that contain only one column. The transpose of a vector is therefore a matrix with only one row. Sometimes we

define a vector by writing out its elements in the text inline as a row matrix, then using the transpose operator to turn it into a standard column vector, e.g., $\mathbf{x} = [x_1, x_2, x_3]^\top$.

A scalar can be thought of as a matrix with only a single entry. From this, we can see that a scalar is its own transpose: $a = a^\top$.

We can add matrices to each other, as long as they have the same shape, just by adding their corresponding elements: $\mathbf{C} = \mathbf{A} + \mathbf{B}$ where $C_{i,j} = A_{i,j} + B_{i,j}$.

We can also add a scalar to a matrix or multiply a matrix by a scalar, just by performing that operation on each element of a matrix: $\mathbf{D} = a \cdot \mathbf{B} + c$ where $D_{i,j} = a \cdot B_{i,j} + c$.

In the context of deep learning, we also use some less conventional notation. We allow the addition of matrix and a vector, yielding another matrix: $\mathbf{C} = \mathbf{A} + \mathbf{b}$, where $C_{i,j} = A_{i,j} + b_j$. In other words, the vector $\mathbf{b}$ is added to each row of the matrix. This shorthand eliminates the need to define a matrix with $\mathbf{b}$ copied into each row before doing the addition. This implicit copying of $\mathbf{b}$ to many locations is called **broadcasting**.

2.2 Multiplying Matrices and Vectors

One of the most important operations involving matrices is multiplication of two matrices. The **matrix product** of matrices $\mathbf{A}$ and $\mathbf{B}$ is a third matrix $\mathbf{C}$. In order for this product to be defined, $\mathbf{A}$ must have the same number of columns as $\mathbf{B}$ has rows. If $\mathbf{A}$ is of shape $m \times n$ and $\mathbf{B}$ is of shape $n \times p$, then $\mathbf{C}$ is of shape $m \times p$. We can write the matrix product just by placing two or more matrices together, e.g.

$$\mathbf{C} = \mathbf{AB}. \quad (2.4)$$

The product operation is defined by

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}. \quad (2.5)$$

Note that the standard product of two matrices is *not* just a matrix containing the product of the individual elements. Such an operation exists and is called the **element-wise product** or **Hadamard product**, and is denoted as $\mathbf{A} \odot \mathbf{B}$.

The **dot product** between two vectors $\mathbf{x}$ and $\mathbf{y}$ of the same dimensionality is the matrix product $\mathbf{x}^\top \mathbf{y}$. We can think of the matrix product $\mathbf{C} = \mathbf{AB}$ as computing $C_{i,j}$ as the dot product between row i of $\mathbf{A}$ and column j of $\mathbf{B}$.

Matrix product operations have many useful properties that make mathematical analysis of matrices more convenient. For example, matrix multiplication is distributive:

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}. \quad (2.6)$$

It is also associative:

$$\mathbf{A}(\mathbf{BC}) = (\mathbf{AB})\mathbf{C}. \quad (2.7)$$

Matrix multiplication is *not* commutative (the condition $\mathbf{AB} = \mathbf{BA}$ does not always hold), unlike scalar multiplication. However, the dot product between two vectors is commutative:

$$\mathbf{x}^\top \mathbf{y} = \mathbf{y}^\top \mathbf{x}. \quad (2.8)$$

The transpose of a matrix product has a simple form:

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top. \quad (2.9)$$

This allows us to demonstrate equation 2.8, by exploiting the fact that the value of such a product is a scalar and therefore equal to its own transpose:

$$\mathbf{x}^\top \mathbf{y} = \left(\mathbf{x}^\top \mathbf{y} \right)^\top = \mathbf{y}^\top \mathbf{x}. \quad (2.10)$$

Since the focus of this textbook is not linear algebra, we do not attempt to develop a comprehensive list of useful properties of the matrix product here, but the reader should be aware that many more exist.

We now know enough linear algebra notation to write down a system of linear equations:

$$\mathbf{Ax} = \mathbf{b} \quad (2.11)$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a known matrix, $\mathbf{b} \in \mathbb{R}^m$ is a known vector, and $\mathbf{x} \in \mathbb{R}^n$ is a vector of unknown variables we would like to solve for. Each element x_i of $\mathbf{x}$ is one of these unknown variables. Each row of $\mathbf{A}$ and each element of $\mathbf{b}$ provide another constraint. We can rewrite equation 2.11 as:

$$\mathbf{A}_{1,:}\mathbf{x} = b_1 \quad (2.12)$$

$$\mathbf{A}_{2,:}\mathbf{x} = b_2 \quad (2.13)$$

$$\dots \quad (2.14)$$

$$\mathbf{A}_{m,:}\mathbf{x} = b_m \quad (2.15)$$

or, even more explicitly, as:

$$\mathbf{A}_{1,1}x_1 + \mathbf{A}_{1,2}x_2 + \dots + \mathbf{A}_{1,n}x_n = b_1 \quad (2.16)$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Figure 2.2: **Example identity matrix:** This is $\mathbf{I}_3$.

$$\mathbf{A}_{2,1}x_1 + \mathbf{A}_{2,2}x_2 + \cdots + \mathbf{A}_{2,n}x_n = b_2 \quad (2.17)$$

$$\dots \quad (2.18)$$

$$\mathbf{A}_{m,1}x_1 + \mathbf{A}_{m,2}x_2 + \cdots + \mathbf{A}_{m,n}x_n = b_m. \quad (2.19)$$

Matrix-vector product notation provides a more compact representation for equations of this form.

2.3 Identity and Inverse Matrices

Linear algebra offers a powerful tool called **matrix inversion** that allows us to analytically solve equation 2.11 for many values of $\mathbf{A}$.

To describe matrix inversion, we first need to define the concept of an **identity matrix**. An identity matrix is a matrix that does not change any vector when we multiply that vector by that matrix. We denote the identity matrix that preserves n -dimensional vectors as $\mathbf{I}_n$. Formally, $\mathbf{I}_n \in \mathbb{R}^{n \times n}$, and

$$\forall \mathbf{x} \in \mathbb{R}^n, \mathbf{I}_n \mathbf{x} = \mathbf{x}. \quad (2.20)$$

The structure of the identity matrix is simple: all of the entries along the main diagonal are 1, while all of the other entries are zero. See figure 2.2 for an example.

The **matrix inverse** of $\mathbf{A}$ is denoted as $\mathbf{A}^{-1}$, and it is defined as the matrix such that

$$\mathbf{A}^{-1} \mathbf{A} = \mathbf{I}_n. \quad (2.21)$$

We can now solve equation 2.11 by the following steps:

$$\mathbf{A} \mathbf{x} = \mathbf{b} \quad (2.22)$$

$$\mathbf{A}^{-1} \mathbf{A} \mathbf{x} = \mathbf{A}^{-1} \mathbf{b} \quad (2.23)$$

$$\mathbf{I}_n \mathbf{x} = \mathbf{A}^{-1} \mathbf{b} \quad (2.24)$$

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}. \quad (2.25)$$

Of course, this process depends on it being possible to find $\mathbf{A}^{-1}$. We discuss the conditions for the existence of $\mathbf{A}^{-1}$ in the following section.

When $\mathbf{A}^{-1}$ exists, several different algorithms exist for finding it in closed form. In theory, the same inverse matrix can then be used to solve the equation many times for different values of $\mathbf{b}$. However, $\mathbf{A}^{-1}$ is primarily useful as a theoretical tool, and should not actually be used in practice for most software applications. Because $\mathbf{A}^{-1}$ can be represented with only limited precision on a digital computer, algorithms that make use of the value of $\mathbf{b}$ can usually obtain more accurate estimates of $\mathbf{x}$.

2.4 Linear Dependence and Span

In order for $\mathbf{A}^{-1}$ to exist, equation 2.11 must have exactly one solution for every value of $\mathbf{b}$. However, it is also possible for the system of equations to have no solutions or infinitely many solutions for some values of $\mathbf{b}$. It is not possible to have more than one but less than infinitely many solutions for a particular $\mathbf{b}$; if both $\mathbf{x}$ and $\mathbf{y}$ are solutions then

$$\mathbf{z} = \alpha\mathbf{x} + (1 - \alpha)\mathbf{y} \quad (2.26)$$

is also a solution for any real α .

To analyze how many solutions the equation has, we can think of the columns of $\mathbf{A}$ as specifying different directions we can travel from the **origin** (the point specified by the vector of all zeros), and determine how many ways there are of reaching $\mathbf{b}$. In this view, each element of $\mathbf{x}$ specifies how far we should travel in each of these directions, with x_i specifying how far to move in the direction of column i :

$$\mathbf{Ax} = \sum_i x_i \mathbf{A}_{:,i}. \quad (2.27)$$

In general, this kind of operation is called a **linear combination**. Formally, a linear combination of some set of vectors $\{\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(n)}\}$ is given by multiplying each vector $\mathbf{v}^{(i)}$ by a corresponding scalar coefficient and adding the results:

$$\sum_i c_i \mathbf{v}^{(i)}. \quad (2.28)$$

The **span** of a set of vectors is the set of all points obtainable by linear combination of the original vectors.

Determining whether $\mathbf{Ax} = \mathbf{b}$ has a solution thus amounts to testing whether $\mathbf{b}$ is in the span of the columns of $\mathbf{A}$. This particular span is known as the **column space** or the **range** of $\mathbf{A}$.

In order for the system $\mathbf{Ax} = \mathbf{b}$ to have a solution for all values of $\mathbf{b} \in \mathbb{R}^m$, we therefore require that the column space of $\mathbf{A}$ be all of $\mathbb{R}^m$. If any point in $\mathbb{R}^m$ is excluded from the column space, that point is a potential value of $\mathbf{b}$ that has no solution. The requirement that the column space of $\mathbf{A}$ be all of $\mathbb{R}^m$ implies immediately that $\mathbf{A}$ must have at least m columns, i.e., $n \geq m$. Otherwise, the dimensionality of the column space would be less than m . For example, consider a 3×2 matrix. The target $\mathbf{b}$ is 3-D, but $\mathbf{x}$ is only 2-D, so modifying the value of $\mathbf{x}$ at best allows us to trace out a 2-D plane within $\mathbb{R}^3$. The equation has a solution if and only if $\mathbf{b}$ lies on that plane.

Having $n \geq m$ is only a necessary condition for every point to have a solution. It is not a sufficient condition, because it is possible for some of the columns to be redundant. Consider a 2×2 matrix where both of the columns are identical. This has the same column space as a 2×1 matrix containing only one copy of the replicated column. In other words, the column space is still just a line, and fails to encompass all of $\mathbb{R}^2$, even though there are two columns.

Formally, this kind of redundancy is known as **linear dependence**. A set of vectors is **linearly independent** if no vector in the set is a linear combination of the other vectors. If we add a vector to a set that is a linear combination of the other vectors in the set, the new vector does not add any points to the set's span. This means that for the column space of the matrix to encompass all of $\mathbb{R}^m$, the matrix must contain at least one set of m linearly independent columns. This condition is both necessary and sufficient for equation 2.11 to have a solution for every value of $\mathbf{b}$. Note that the requirement is for a set to have exactly m linear independent columns, not at least m . No set of m -dimensional vectors can have more than m mutually linearly independent columns, but a matrix with more than m columns may have more than one such set.

In order for the matrix to have an inverse, we additionally need to ensure that equation 2.11 has *at most* one solution for each value of $\mathbf{b}$. To do so, we need to ensure that the matrix has at most m columns. Otherwise there is more than one way of parametrizing each solution.

Together, this means that the matrix must be **square**, that is, we require that $m = n$ and that all of the columns must be linearly independent. A square matrix with linearly dependent columns is known as **singular**.

If $\mathbf{A}$ is not square or is square but singular, it can still be possible to solve the equation. However, we can not use the method of matrix inversion to find the

solution.

So far we have discussed matrix inverses as being multiplied on the left. It is also possible to define an inverse that is multiplied on the right:

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}. \quad (2.29)$$

For square matrices, the left inverse and right inverse are equal.

2.5 Norms

Sometimes we need to measure the size of a vector. In machine learning, we usually measure the size of vectors using a function called a **norm**. Formally, the L^p norm is given by

$$\|\mathbf{x}\|_p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}} \quad (2.30)$$

for $p \in \mathbb{R}, p \geq 1$.

Norms, including the L^p norm, are functions mapping vectors to non-negative values. On an intuitive level, the norm of a vector $\mathbf{x}$ measures the distance from the origin to the point $\mathbf{x}$. More rigorously, a norm is any function f that satisfies the following properties:

- $f(\mathbf{x}) = 0 \Rightarrow \mathbf{x} = \mathbf{0}$
- $f(\mathbf{x} + \mathbf{y}) \leq f(\mathbf{x}) + f(\mathbf{y})$ (the **triangle inequality**)
- $\forall \alpha \in \mathbb{R}, f(\alpha \mathbf{x}) = |\alpha|f(\mathbf{x})$

The L^2 norm, with $p = 2$, is known as the **Euclidean norm**. It is simply the Euclidean distance from the origin to the point identified by $\mathbf{x}$. The L^2 norm is used so frequently in machine learning that it is often denoted simply as $\|\mathbf{x}\|$, with the subscript 2 omitted. It is also common to measure the size of a vector using the squared L^2 norm, which can be calculated simply as $\mathbf{x}^\top \mathbf{x}$.

The squared L^2 norm is more convenient to work with mathematically and computationally than the L^2 norm itself. For example, the derivatives of the squared L^2 norm with respect to each element of $\mathbf{x}$ each depend only on the corresponding element of $\mathbf{x}$, while all of the derivatives of the L^2 norm depend on the entire vector. In many contexts, the squared L^2 norm may be undesirable because it increases very slowly near the origin. In several machine learning

applications, it is important to discriminate between elements that are exactly zero and elements that are small but nonzero. In these cases, we turn to a function that grows at the same rate in all locations, but retains mathematical simplicity: the L^1 norm. The L^1 norm may be simplified to

$$\|\mathbf{x}\|_1 = \sum_i |x_i|. \quad (2.31)$$

The L^1 norm is commonly used in machine learning when the difference between zero and nonzero elements is very important. Every time an element of $\mathbf{x}$ moves away from 0 by ϵ , the L^1 norm increases by ϵ .

We sometimes measure the size of the vector by counting its number of nonzero elements. Some authors refer to this function as the “ L^0 norm,” but this is incorrect terminology. The number of non-zero entries in a vector is not a norm, because scaling the vector by α does not change the number of nonzero entries. The L^1 norm is often used as a substitute for the number of nonzero entries.

One other norm that commonly arises in machine learning is the L^∞ norm, also known as the **max norm**. This norm simplifies to the absolute value of the element with the largest magnitude in the vector,

$$\|\mathbf{x}\|_\infty = \max_i |x_i|. \quad (2.32)$$

Sometimes we may also wish to measure the size of a matrix. In the context of deep learning, the most common way to do this is with the otherwise obscure **Frobenius norm**:

$$\|A\|_F = \sqrt{\sum_{i,j} A_{i,j}^2}, \quad (2.33)$$

which is analogous to the L^2 norm of a vector.

The dot product of two vectors can be rewritten in terms of norms. Specifically,

$$\mathbf{x}^\top \mathbf{y} = \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \cos \theta \quad (2.34)$$

where θ is the angle between $\mathbf{x}$ and $\mathbf{y}$.

2.6 Special Kinds of Matrices and Vectors

Some special kinds of matrices and vectors are particularly useful.

Diagonal matrices consist mostly of zeros and have non-zero entries only along the main diagonal. Formally, a matrix $\mathbf{D}$ is diagonal if and only if $D_{i,j} = 0$ for

all $i \neq j$. We have already seen one example of a diagonal matrix: the identity matrix, where all of the diagonal entries are 1. We write $\text{diag}(\mathbf{v})$ to denote a square diagonal matrix whose diagonal entries are given by the entries of the vector $\mathbf{v}$. Diagonal matrices are of interest in part because multiplying by a diagonal matrix is very computationally efficient. To compute $\text{diag}(\mathbf{v})\mathbf{x}$, we only need to scale each element x_i by v_i . In other words, $\text{diag}(\mathbf{v})\mathbf{x} = \mathbf{v} \odot \mathbf{x}$. Inverting a square diagonal matrix is also efficient. The inverse exists only if every diagonal entry is nonzero, and in that case, $\text{diag}(\mathbf{v})^{-1} = \text{diag}([1/v_1, \dots, 1/v_n]^\top)$. In many cases, we may derive some very general machine learning algorithm in terms of arbitrary matrices, but obtain a less expensive (and less descriptive) algorithm by restricting some matrices to be diagonal.

Not all diagonal matrices need be square. It is possible to construct a rectangular diagonal matrix. Non-square diagonal matrices do not have inverses but it is still possible to multiply by them cheaply. For a non-square diagonal matrix $\mathbf{D}$, the product $\mathbf{D}\mathbf{x}$ will involve scaling each element of $\mathbf{x}$, and either concatenating some zeros to the result if $\mathbf{D}$ is taller than it is wide, or discarding some of the last elements of the vector if $\mathbf{D}$ is wider than it is tall.

A **symmetric** matrix is any matrix that is equal to its own transpose:

$$\mathbf{A} = \mathbf{A}^\top. \quad (2.35)$$

Symmetric matrices often arise when the entries are generated by some function of two arguments that does not depend on the order of the arguments. For example, if $\mathbf{A}$ is a matrix of distance measurements, with $\mathbf{A}_{i,j}$ giving the distance from point i to point j , then $\mathbf{A}_{i,j} = \mathbf{A}_{j,i}$ because distance functions are symmetric.

A **unit vector** is a vector with **unit norm**:

$$\|\mathbf{x}\|_2 = 1. \quad (2.36)$$

A vector $\mathbf{x}$ and a vector $\mathbf{y}$ are **orthogonal** to each other if $\mathbf{x}^\top \mathbf{y} = 0$. If both vectors have nonzero norm, this means that they are at a 90 degree angle to each other. In $\mathbb{R}^n$, at most n vectors may be mutually orthogonal with nonzero norm. If the vectors are not only orthogonal but also have unit norm, we call them **orthonormal**.

An **orthogonal matrix** is a square matrix whose rows are mutually orthonormal and whose columns are mutually orthonormal:

$$\mathbf{A}^\top \mathbf{A} = \mathbf{A} \mathbf{A}^\top = \mathbf{I}. \quad (2.37)$$

This implies that

$$\mathbf{A}^{-1} = \mathbf{A}^\top, \quad (2.38)$$

so orthogonal matrices are of interest because their inverse is very cheap to compute. Pay careful attention to the definition of orthogonal matrices. Counterintuitively, their rows are not merely orthogonal but fully orthonormal. There is no special term for a matrix whose rows or columns are orthogonal but not orthonormal.

2.7 Eigendecomposition

Many mathematical objects can be understood better by breaking them into constituent parts, or finding some properties of them that are universal, not caused by the way we choose to represent them.

For example, integers can be decomposed into prime factors. The way we represent the number 12 will change depending on whether we write it in base ten or in binary, but it will always be true that $12 = 2 \times 2 \times 3$. From this representation we can conclude useful properties, such as that 12 is not divisible by 5, or that any integer multiple of 12 will be divisible by 3.

Much as we can discover something about the true nature of an integer by decomposing it into prime factors, we can also decompose matrices in ways that show us information about their functional properties that is not obvious from the representation of the matrix as an array of elements.

One of the most widely used kinds of matrix decomposition is called **eigendecomposition**, in which we decompose a matrix into a set of eigenvectors and eigenvalues.

An **eigenvector** of a square matrix $\mathbf{A}$ is a non-zero vector $\mathbf{v}$ such that multiplication by $\mathbf{A}$ alters only the scale of $\mathbf{v}$:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}. \quad (2.39)$$

The scalar λ is known as the **eigenvalue** corresponding to this eigenvector. (One can also find a **left eigenvector** such that $\mathbf{v}^\top \mathbf{A} = \lambda \mathbf{v}^\top$, but we are usually concerned with right eigenvectors).

If $\mathbf{v}$ is an eigenvector of $\mathbf{A}$, then so is any rescaled vector $s\mathbf{v}$ for $s \in \mathbb{R}, s \neq 0$. Moreover, $s\mathbf{v}$ still has the same eigenvalue. For this reason, we usually only look for unit eigenvectors.

Suppose that a matrix $\mathbf{A}$ has n linearly independent eigenvectors, $\{\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(n)}\}$, with corresponding eigenvalues $\{\lambda_1, \dots, \lambda_n\}$. We may concatenate all of the

Effect of eigenvectors and eigenvalues

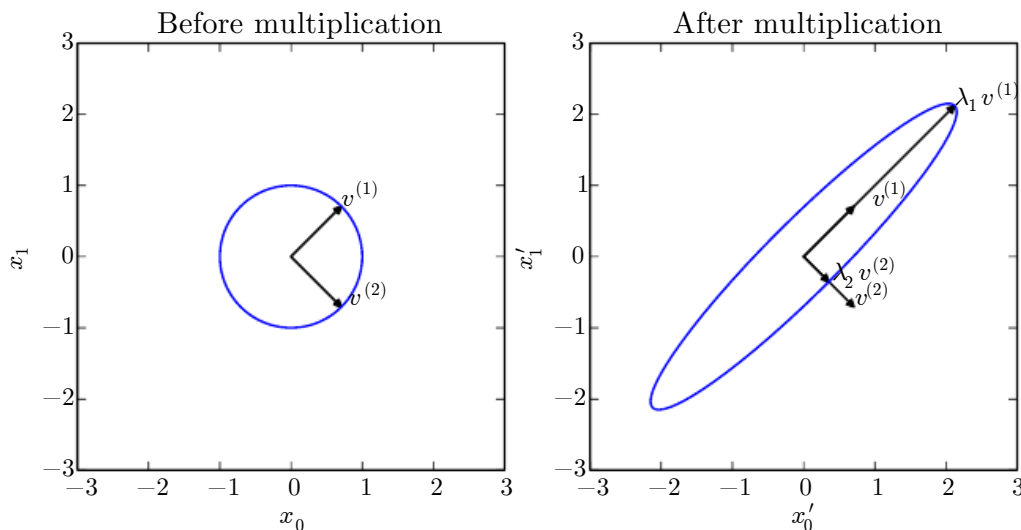


Figure 2.3: An example of the effect of eigenvectors and eigenvalues. Here, we have a matrix $\mathbf{A}$ with two orthonormal eigenvectors, $\mathbf{v}^{(1)}$ with eigenvalue λ_1 and $\mathbf{v}^{(2)}$ with eigenvalue λ_2 . (Left) We plot the set of all unit vectors $\mathbf{u} \in \mathbb{R}^2$ as a unit circle. (Right) We plot the set of all points $\mathbf{A}\mathbf{u}$. By observing the way that $\mathbf{A}$ distorts the unit circle, we can see that it scales space in direction $\mathbf{v}^{(i)}$ by λ_i .

eigenvectors to form a matrix $\mathbf{V}$ with one eigenvector per column: $\mathbf{V} = [\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(n)}]$. Likewise, we can concatenate the eigenvalues to form a vector $\boldsymbol{\lambda} = [\lambda_1, \dots, \lambda_n]^\top$. The **eigendecomposition** of $\mathbf{A}$ is then given by

$$\mathbf{A} = \mathbf{V} \text{diag}(\boldsymbol{\lambda}) \mathbf{V}^{-1}. \quad (2.40)$$

We have seen that *constructing* matrices with specific eigenvalues and eigenvectors allows us to stretch space in desired directions. However, we often want to **decompose** matrices into their eigenvalues and eigenvectors. Doing so can help us to analyze certain properties of the matrix, much as decomposing an integer into its prime factors can help us understand the behavior of that integer.

Not every matrix can be decomposed into eigenvalues and eigenvectors. In some

cases, the decomposition exists, but may involve complex rather than real numbers. Fortunately, in this book, we usually need to decompose only a specific class of matrices that have a simple decomposition. Specifically, every real symmetric matrix can be decomposed into an expression using only real-valued eigenvectors and eigenvalues:

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top, \quad (2.41)$$

where $\mathbf{Q}$ is an orthogonal matrix composed of eigenvectors of $\mathbf{A}$, and $\mathbf{\Lambda}$ is a diagonal matrix. The eigenvalue $\Lambda_{i,i}$ is associated with the eigenvector in column i of $\mathbf{Q}$, denoted as $\mathbf{Q}_{:,i}$. Because $\mathbf{Q}$ is an orthogonal matrix, we can think of $\mathbf{A}$ as scaling space by λ_i in direction $\mathbf{v}^{(i)}$. See figure 2.3 for an example.

While any real symmetric matrix $\mathbf{A}$ is guaranteed to have an eigendecomposition, the eigendecomposition may not be unique. If any two or more eigenvectors share the same eigenvalue, then any set of orthogonal vectors lying in their span are also eigenvectors with that eigenvalue, and we could equivalently choose a $\mathbf{Q}$ using those eigenvectors instead. By convention, we usually sort the entries of $\mathbf{\Lambda}$ in descending order. Under this convention, the eigendecomposition is unique only if all of the eigenvalues are unique.

The eigendecomposition of a matrix tells us many useful facts about the matrix. The matrix is singular if and only if any of the eigenvalues are zero. The eigendecomposition of a real symmetric matrix can also be used to optimize quadratic expressions of the form $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x}$ subject to $\|\mathbf{x}\|_2 = 1$. Whenever $\mathbf{x}$ is equal to an eigenvector of $\mathbf{A}$, f takes on the value of the corresponding eigenvalue. The maximum value of f within the constraint region is the maximum eigenvalue and its minimum value within the constraint region is the minimum eigenvalue.

A matrix whose eigenvalues are all positive is called **positive definite**. A matrix whose eigenvalues are all positive or zero-valued is called **positive semidefinite**. Likewise, if all eigenvalues are negative, the matrix is **negative definite**, and if all eigenvalues are negative or zero-valued, it is **negative semidefinite**. Positive semidefinite matrices are interesting because they guarantee that $\forall \mathbf{x}, \mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$. Positive definite matrices additionally guarantee that $\mathbf{x}^\top \mathbf{A} \mathbf{x} = 0 \Rightarrow \mathbf{x} = \mathbf{0}$.

2.8 Singular Value Decomposition

In section 2.7, we saw how to decompose a matrix into eigenvectors and eigenvalues. The **singular value decomposition** (SVD) provides another way to factorize a matrix, into **singular vectors** and **singular values**. The SVD allows us to discover some of the same kind of information as the eigendecomposition. However,

the SVD is more generally applicable. Every real matrix has a singular value decomposition, but the same is not true of the eigenvalue decomposition. For example, if a matrix is not square, the eigendecomposition is not defined, and we must use a singular value decomposition instead.

Recall that the eigendecomposition involves analyzing a matrix $\mathbf{A}$ to discover a matrix $\mathbf{V}$ of eigenvectors and a vector of eigenvalues $\boldsymbol{\lambda}$ such that we can rewrite $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{V} \text{diag}(\boldsymbol{\lambda}) \mathbf{V}^{-1}. \quad (2.42)$$

The singular value decomposition is similar, except this time we will write $\mathbf{A}$ as a product of three matrices:

$$\mathbf{A} = \mathbf{U} \mathbf{D} \mathbf{V}^{\top}. \quad (2.43)$$

Suppose that $\mathbf{A}$ is an $m \times n$ matrix. Then $\mathbf{U}$ is defined to be an $m \times m$ matrix, $\mathbf{D}$ to be an $m \times n$ matrix, and $\mathbf{V}$ to be an $n \times n$ matrix.

Each of these matrices is defined to have a special structure. The matrices $\mathbf{U}$ and $\mathbf{V}$ are both defined to be orthogonal matrices. The matrix $\mathbf{D}$ is defined to be a diagonal matrix. Note that $\mathbf{D}$ is not necessarily square.

The elements along the diagonal of $\mathbf{D}$ are known as the **singular values** of the matrix $\mathbf{A}$. The columns of $\mathbf{U}$ are known as the **left-singular vectors**. The columns of $\mathbf{V}$ are known as the **right-singular vectors**.

We can actually interpret the singular value decomposition of $\mathbf{A}$ in terms of the eigendecomposition of functions of $\mathbf{A}$. The left-singular vectors of $\mathbf{A}$ are the eigenvectors of $\mathbf{A} \mathbf{A}^{\top}$. The right-singular vectors of $\mathbf{A}$ are the eigenvectors of $\mathbf{A}^{\top} \mathbf{A}$. The non-zero singular values of $\mathbf{A}$ are the square roots of the eigenvalues of $\mathbf{A}^{\top} \mathbf{A}$. The same is true for $\mathbf{A} \mathbf{A}^{\top}$.

Perhaps the most useful feature of the SVD is that we can use it to partially generalize matrix inversion to non-square matrices, as we will see in the next section.

2.9 The Moore-Penrose Pseudoinverse

Matrix inversion is not defined for matrices that are not square. Suppose we want to make a left-inverse $\mathbf{B}$ of a matrix $\mathbf{A}$, so that we can solve a linear equation

$$\mathbf{A} \mathbf{x} = \mathbf{y} \quad (2.44)$$

by left-multiplying each side to obtain

$$\mathbf{x} = \mathbf{B}\mathbf{y}. \quad (2.45)$$

Depending on the structure of the problem, it may not be possible to design a unique mapping from $\mathbf{A}$ to $\mathbf{B}$.

If $\mathbf{A}$ is taller than it is wide, then it is possible for this equation to have no solution. If $\mathbf{A}$ is wider than it is tall, then there could be multiple possible solutions.

The **Moore-Penrose pseudoinverse** allows us to make some headway in these cases. The pseudoinverse of $\mathbf{A}$ is defined as a matrix

$$\mathbf{A}^+ = \lim_{\alpha \searrow 0} (\mathbf{A}^\top \mathbf{A} + \alpha \mathbf{I})^{-1} \mathbf{A}^\top. \quad (2.46)$$

Practical algorithms for computing the pseudoinverse are not based on this definition, but rather the formula

$$\mathbf{A}^+ = \mathbf{V}\mathbf{D}^+\mathbf{U}^\top, \quad (2.47)$$

where $\mathbf{U}$, $\mathbf{D}$ and $\mathbf{V}$ are the singular value decomposition of $\mathbf{A}$, and the pseudoinverse $\mathbf{D}^+$ of a diagonal matrix $\mathbf{D}$ is obtained by taking the reciprocal of its non-zero elements then taking the transpose of the resulting matrix.

When $\mathbf{A}$ has more columns than rows, then solving a linear equation using the pseudoinverse provides one of the many possible solutions. Specifically, it provides the solution $\mathbf{x} = \mathbf{A}^+ \mathbf{y}$ with minimal Euclidean norm $\|\mathbf{x}\|_2$ among all possible solutions.

When $\mathbf{A}$ has more rows than columns, it is possible for there to be no solution. In this case, using the pseudoinverse gives us the $\mathbf{x}$ for which $\mathbf{A}\mathbf{x}$ is as close as possible to $\mathbf{y}$ in terms of Euclidean norm $\|\mathbf{A}\mathbf{x} - \mathbf{y}\|_2$.

2.10 The Trace Operator

The trace operator gives the sum of all of the diagonal entries of a matrix:

$$\text{Tr}(\mathbf{A}) = \sum_i \mathbf{A}_{i,i}. \quad (2.48)$$

The trace operator is useful for a variety of reasons. Some operations that are difficult to specify without resorting to summation notation can be specified using

matrix products and the trace operator. For example, the trace operator provides an alternative way of writing the Frobenius norm of a matrix:

$$\|A\|_F = \sqrt{\text{Tr}(\mathbf{A}\mathbf{A}^\top)}. \quad (2.49)$$

Writing an expression in terms of the trace operator opens up opportunities to manipulate the expression using many useful identities. For example, the trace operator is invariant to the transpose operator:

$$\text{Tr}(\mathbf{A}) = \text{Tr}(\mathbf{A}^\top). \quad (2.50)$$

The trace of a square matrix composed of many factors is also invariant to moving the last factor into the first position, if the shapes of the corresponding matrices allow the resulting product to be defined:

$$\text{Tr}(\mathbf{ABC}) = \text{Tr}(\mathbf{CAB}) = \text{Tr}(\mathbf{BCA}) \quad (2.51)$$

or more generally,

$$\text{Tr}\left(\prod_{i=1}^n \mathbf{F}^{(i)}\right) = \text{Tr}\left(\mathbf{F}^{(n)} \prod_{i=1}^{n-1} \mathbf{F}^{(i)}\right). \quad (2.52)$$

This invariance to cyclic permutation holds even if the resulting product has a different shape. For example, for $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times m}$, we have

$$\text{Tr}(\mathbf{AB}) = \text{Tr}(\mathbf{BA}) \quad (2.53)$$

even though $\mathbf{AB} \in \mathbb{R}^{m \times m}$ and $\mathbf{BA} \in \mathbb{R}^{n \times n}$.

Another useful fact to keep in mind is that a scalar is its own trace: $a = \text{Tr}(a)$.

2.11 The Determinant

The determinant of a square matrix, denoted $\det(\mathbf{A})$, is a function mapping matrices to real scalars. The determinant is equal to the product of all the eigenvalues of the matrix. The absolute value of the determinant can be thought of as a measure of how much multiplication by the matrix expands or contracts space. If the determinant is 0, then space is contracted completely along at least one dimension, causing it to lose all of its volume. If the determinant is 1, then the transformation preserves volume.

2.12 Example: Principal Components Analysis

One simple machine learning algorithm, **principal components analysis** or PCA can be derived using only knowledge of basic linear algebra.

Suppose we have a collection of m points $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ in $\mathbb{R}^n$. Suppose we would like to apply lossy compression to these points. Lossy compression means storing the points in a way that requires less memory but may lose some precision. We would like to lose as little precision as possible.

One way we can encode these points is to represent a lower-dimensional version of them. For each point $\mathbf{x}^{(i)} \in \mathbb{R}^n$ we will find a corresponding code vector $\mathbf{c}^{(i)} \in \mathbb{R}^l$. If l is smaller than n , it will take less memory to store the code points than the original data. We will want to find some encoding function that produces the code for an input, $f(\mathbf{x}) = \mathbf{c}$, and a decoding function that produces the reconstructed input given its code, $\mathbf{x} \approx g(f(\mathbf{x}))$.

PCA is defined by our choice of the decoding function. Specifically, to make the decoder very simple, we choose to use matrix multiplication to map the code back into $\mathbb{R}^n$. Let $g(\mathbf{c}) = \mathbf{D}\mathbf{c}$, where $\mathbf{D} \in \mathbb{R}^{n \times l}$ is the matrix defining the decoding.

Computing the optimal code for this decoder could be a difficult problem. To keep the encoding problem easy, PCA constrains the columns of $\mathbf{D}$ to be orthogonal to each other. (Note that $\mathbf{D}$ is still not technically “an orthogonal matrix” unless $l = n$)

With the problem as described so far, many solutions are possible, because we can increase the scale of $\mathbf{D}_{:,i}$ if we decrease c_i proportionally for all points. To give the problem a unique solution, we constrain all of the columns of $\mathbf{D}$ to have unit norm.

In order to turn this basic idea into an algorithm we can implement, the first thing we need to do is figure out how to generate the optimal code point $\mathbf{c}^*$ for each input point $\mathbf{x}$. One way to do this is to minimize the distance between the input point $\mathbf{x}$ and its reconstruction, $g(\mathbf{c}^*)$. We can measure this distance using a norm. In the principal components algorithm, we use the L^2 norm:

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} \|\mathbf{x} - g(\mathbf{c})\|_2. \quad (2.54)$$

We can switch to the squared L^2 norm instead of the L^2 norm itself, because both are minimized by the same value of $\mathbf{c}$. Both are minimized by the same value of $\mathbf{c}$ because the L^2 norm is non-negative and the squaring operation is

monotonically increasing for non-negative arguments.

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} \|\mathbf{x} - g(\mathbf{c})\|_2^2. \quad (2.55)$$

The function being minimized simplifies to

$$(\mathbf{x} - g(\mathbf{c}))^\top (\mathbf{x} - g(\mathbf{c})) \quad (2.56)$$

(by the definition of the L^2 norm, equation 2.30)

$$= \mathbf{x}^\top \mathbf{x} - \mathbf{x}^\top g(\mathbf{c}) - g(\mathbf{c})^\top \mathbf{x} + g(\mathbf{c})^\top g(\mathbf{c}) \quad (2.57)$$

(by the distributive property)

$$= \mathbf{x}^\top \mathbf{x} - 2\mathbf{x}^\top g(\mathbf{c}) + g(\mathbf{c})^\top g(\mathbf{c}) \quad (2.58)$$

(because the scalar $g(\mathbf{c})^\top \mathbf{x}$ is equal to the transpose of itself).

We can now change the function being minimized again, to omit the first term, since this term does not depend on $\mathbf{c}$:

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} -2\mathbf{x}^\top g(\mathbf{c}) + g(\mathbf{c})^\top g(\mathbf{c}). \quad (2.59)$$

To make further progress, we must substitute in the definition of $g(\mathbf{c})$:

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} -2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{D}^\top \mathbf{D}\mathbf{c} \quad (2.60)$$

$$= \arg \min_{\mathbf{c}} -2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{I}_l \mathbf{c} \quad (2.61)$$

(by the orthogonality and unit norm constraints on $\mathbf{D}$)

$$= \arg \min_{\mathbf{c}} -2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{c} \quad (2.62)$$

We can solve this optimization problem using vector calculus (see section 4.3 if you do not know how to do this):

$$\nabla_{\mathbf{c}}(-2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{c}) = \mathbf{0} \quad (2.63)$$

$$-2\mathbf{D}^\top \mathbf{x} + 2\mathbf{c} = \mathbf{0} \quad (2.64)$$

$$\mathbf{c} = \mathbf{D}^\top \mathbf{x}. \quad (2.65)$$

This makes the algorithm efficient: we can optimally encode $\mathbf{x}$ just using a matrix-vector operation. To encode a vector, we apply the encoder function

$$f(\mathbf{x}) = \mathbf{D}^\top \mathbf{x}. \quad (2.66)$$

Using a further matrix multiplication, we can also define the PCA reconstruction operation:

$$r(\mathbf{x}) = g(f(\mathbf{x})) = \mathbf{D}\mathbf{D}^\top \mathbf{x}. \quad (2.67)$$

Next, we need to choose the encoding matrix $\mathbf{D}$. To do so, we revisit the idea of minimizing the L^2 distance between inputs and reconstructions. Since we will use the same matrix $\mathbf{D}$ to decode all of the points, we can no longer consider the points in isolation. Instead, we must minimize the Frobenius norm of the matrix of errors computed over all dimensions and all points:

$$\mathbf{D}^* = \arg \min_{\mathbf{D}} \sqrt{\sum_{i,j} \left(x_j^{(i)} - r(\mathbf{x}^{(i)})_j \right)^2} \text{ subject to } \mathbf{D}^\top \mathbf{D} = \mathbf{I}_l \quad (2.68)$$

To derive the algorithm for finding $\mathbf{D}^*$, we will start by considering the case where $l = 1$. In this case, $\mathbf{D}$ is just a single vector, $\mathbf{d}$. Substituting equation 2.67 into equation 2.68 and simplifying $\mathbf{D}$ into $\mathbf{d}$, the problem reduces to

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \sum_i \|\mathbf{x}^{(i)} - \mathbf{d}\mathbf{d}^\top \mathbf{x}^{(i)}\|_2^2 \text{ subject to } \|\mathbf{d}\|_2 = 1. \quad (2.69)$$

The above formulation is the most direct way of performing the substitution, but is not the most stylistically pleasing way to write the equation. It places the scalar value $\mathbf{d}^\top \mathbf{x}^{(i)}$ on the right of the vector $\mathbf{d}$. It is more conventional to write scalar coefficients on the left of vector they operate on. We therefore usually write such a formula as

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \sum_i \|\mathbf{x}^{(i)} - \mathbf{d}^\top \mathbf{x}^{(i)} \mathbf{d}\|_2^2 \text{ subject to } \|\mathbf{d}\|_2 = 1, \quad (2.70)$$

or, exploiting the fact that a scalar is its own transpose, as

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \sum_i \|\mathbf{x}^{(i)} - \mathbf{x}^{(i)\top} \mathbf{d} \mathbf{d}\|_2^2 \text{ subject to } \|\mathbf{d}\|_2 = 1. \quad (2.71)$$

The reader should aim to become familiar with such cosmetic rearrangements.

At this point, it can be helpful to rewrite the problem in terms of a single design matrix of examples, rather than as a sum over separate example vectors. This will allow us to use more compact notation. Let $\mathbf{X} \in \mathbb{R}^{m \times n}$ be the matrix defined by stacking all of the vectors describing the points, such that $\mathbf{X}_{i,:} = \mathbf{x}^{(i)\top}$. We can now rewrite the problem as

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \|\mathbf{X} - \mathbf{X} \mathbf{d} \mathbf{d}^\top\|_F^2 \text{ subject to } \mathbf{d}^\top \mathbf{d} = 1. \quad (2.72)$$

Disregarding the constraint for the moment, we can simplify the Frobenius norm portion as follows:

$$\arg \min_{\mathbf{d}} \|\mathbf{X} - \mathbf{X} \mathbf{d} \mathbf{d}^\top\|_F^2 \quad (2.73)$$

$$= \arg \min_{\mathbf{d}} \text{Tr} \left(\left(\mathbf{X} - \mathbf{X} \mathbf{d} \mathbf{d}^\top \right)^\top \left(\mathbf{X} - \mathbf{X} \mathbf{d} \mathbf{d}^\top \right) \right) \quad (2.74)$$

(by equation 2.49)

$$= \arg \min_{\mathbf{d}} \text{Tr}(\mathbf{X}^\top \mathbf{X} - \mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top - \mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X} + \mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \quad (2.75)$$

$$= \arg \min_{\mathbf{d}} \text{Tr}(\mathbf{X}^\top \mathbf{X}) - \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) - \text{Tr}(\mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X}) + \text{Tr}(\mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \quad (2.76)$$

$$= \arg \min_{\mathbf{d}} -\text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) - \text{Tr}(\mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X}) + \text{Tr}(\mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \quad (2.77)$$

(because terms not involving $\mathbf{d}$ do not affect the $\arg \min$)

$$= \arg \min_{\mathbf{d}} -2 \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) + \text{Tr}(\mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \quad (2.78)$$

(because we can cycle the order of the matrices inside a trace, equation 2.52)

$$= \arg \min_{\mathbf{d}} -2 \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) + \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top \mathbf{d} \mathbf{d}^\top) \quad (2.79)$$

(using the same property again)

At this point, we re-introduce the constraint:

$$\arg \min_{\mathbf{d}} -2 \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) + \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top \mathbf{d} \mathbf{d}^\top) \text{ subject to } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.80)$$

$$= \arg \min_{\mathbf{d}} -2 \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) + \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \text{ subject to } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.81)$$

(due to the constraint)

$$= \arg \min_{\mathbf{d}} -\text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \text{ subject to } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.82)$$

$$= \arg \max_{\mathbf{d}} \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \text{ subject to } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.83)$$

$$= \arg \max_{\mathbf{d}} \text{Tr}(\mathbf{d}^\top \mathbf{X}^\top \mathbf{X} \mathbf{d}) \text{ subject to } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.84)$$

This optimization problem may be solved using eigendecomposition. Specifically, the optimal $\mathbf{d}$ is given by the eigenvector of $\mathbf{X}^\top \mathbf{X}$ corresponding to the largest eigenvalue.

This derivation is specific to the case of $l = 1$ and recovers only the first principal component. More generally, when we wish to recover a basis of principal components, the matrix $\mathbf{D}$ is given by the l eigenvectors corresponding to the largest eigenvalues. This may be shown using proof by induction. We recommend writing this proof as an exercise.

Linear algebra is one of the fundamental mathematical disciplines that is necessary to understand deep learning. Another key area of mathematics that is ubiquitous in machine learning is probability theory, presented next.